

---

# コンピュータビジョン特論A/B 現代脳計算論

## Computer Vision A/B Modern Neural Computation

---

Keisuke Yoneda, Kanazawa University  
Email: [k.yoneda@staff.kanazawa-u.ac.jp](mailto:k.yoneda@staff.kanazawa-u.ac.jp)

# 3D Reconstruction

---

162

## □ Basic Technique of 3D Reconstruction

- Stereo Matching
- SfM (Structure From Motion)
- Multi-View Stereo

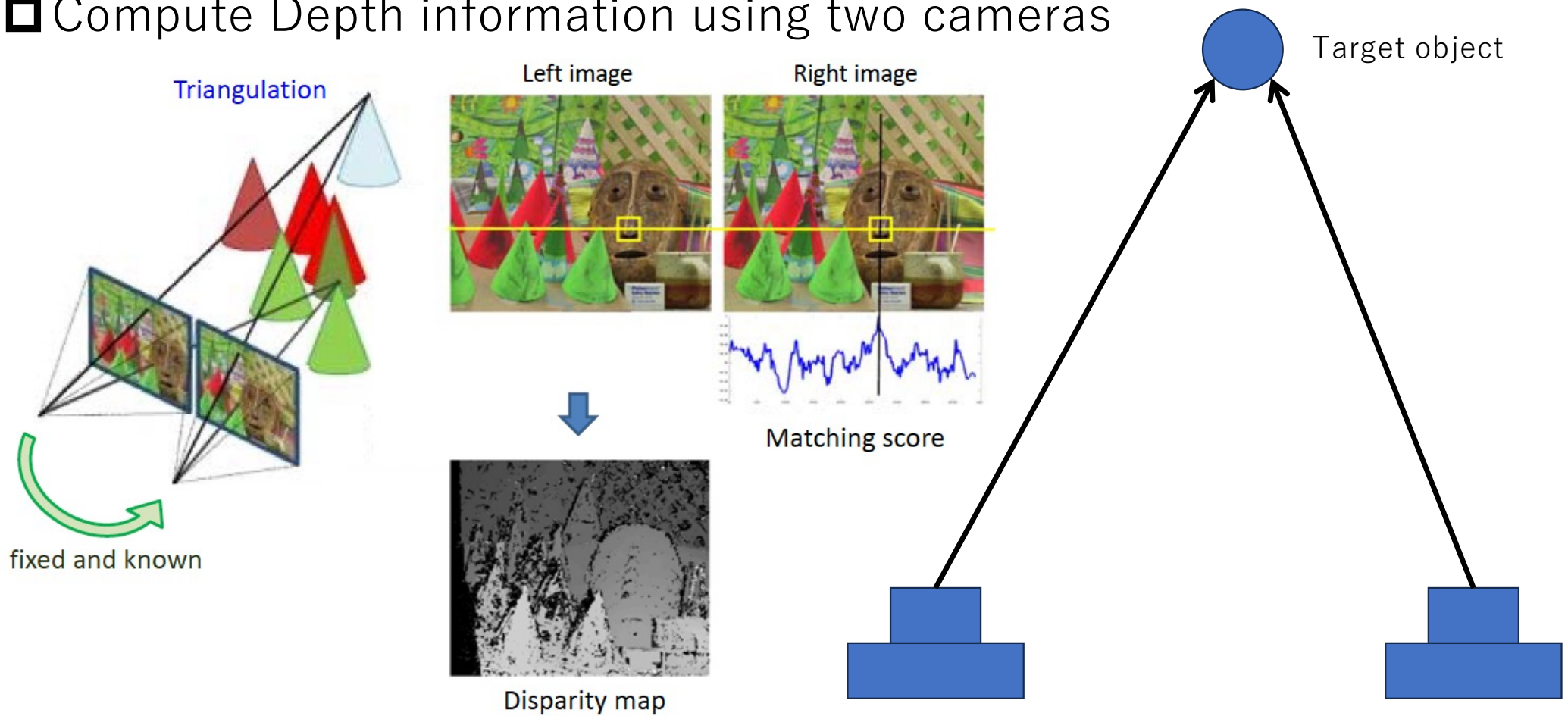
## □ NeRF

- Volume Rendering

# Stereo Vision

163

- Compute Depth information using two cameras



# Epipolar Geometry

164

## □ Focus on object $X$ in left image,

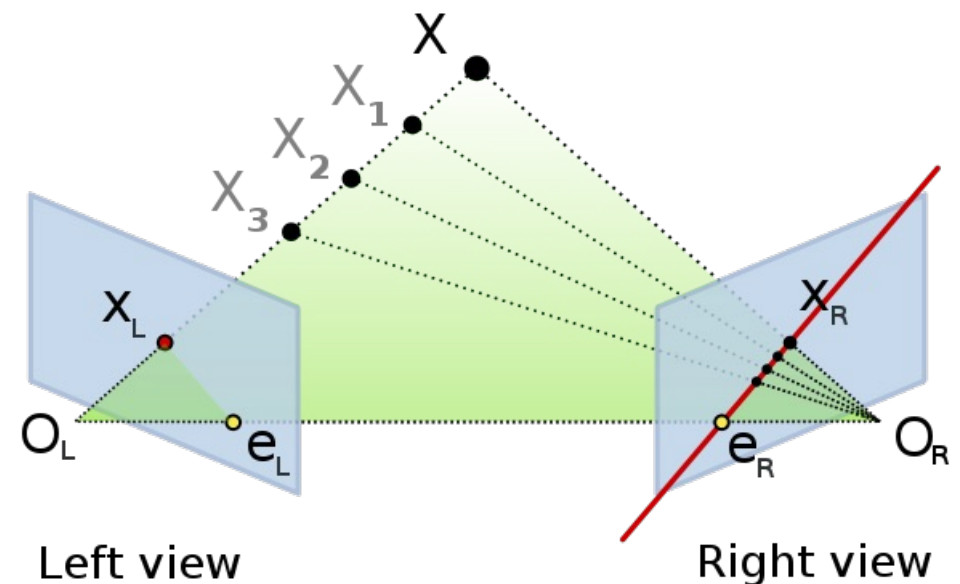
- Object  $X$  can be seen at pixel  $X_L$ 
  - ◆ Cannot obtain distance information using single image
  - ◆ But relative object direction  $O_L - X$  can be obtained

## □ Epipolar Plane

- Green Plane:  $O_L X O_R$

## □ Epipolar Line in right image

- Intersect line for Epipolar plane & Image plane for right camera
- Red Line :  $e_R X_R$
- The object  $X$  should be seen on the epipolar line



<https://ja.wikipedia.org/wiki/%E3%82%A8%E3%83%94%E3%83%9D%E3%83%BC%E3%83%A9%E5%B9%BE%E4%BD%95>

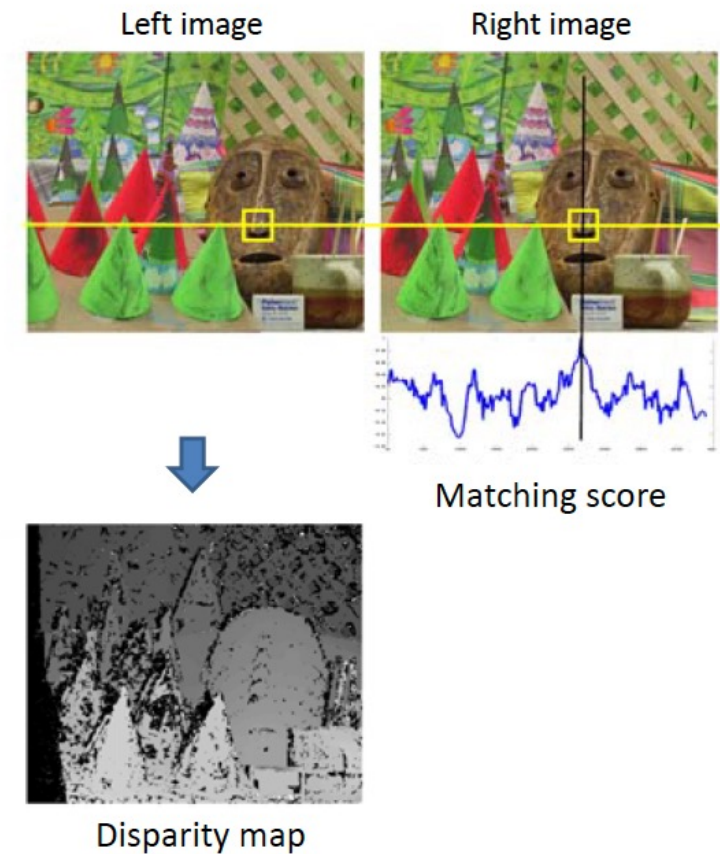
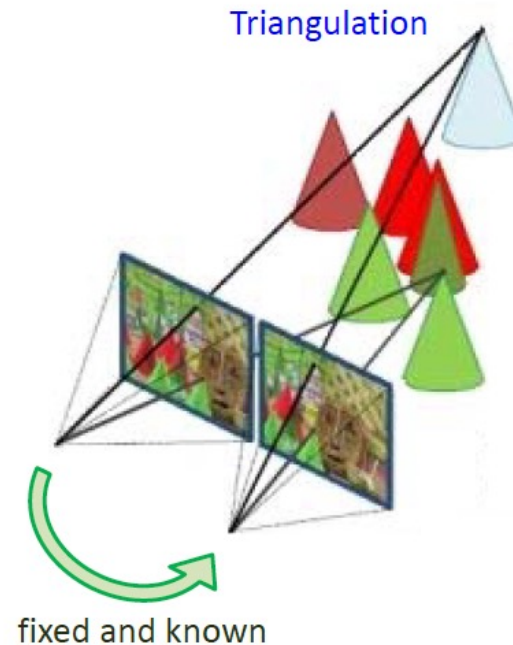
# Stereo Matching

165

## □ Compute depth value for each pixel

1. Rectify Images
2. For each pixel
  1. Find epipolar line
  2. Find corresponded pixel
  3. Compute depth value

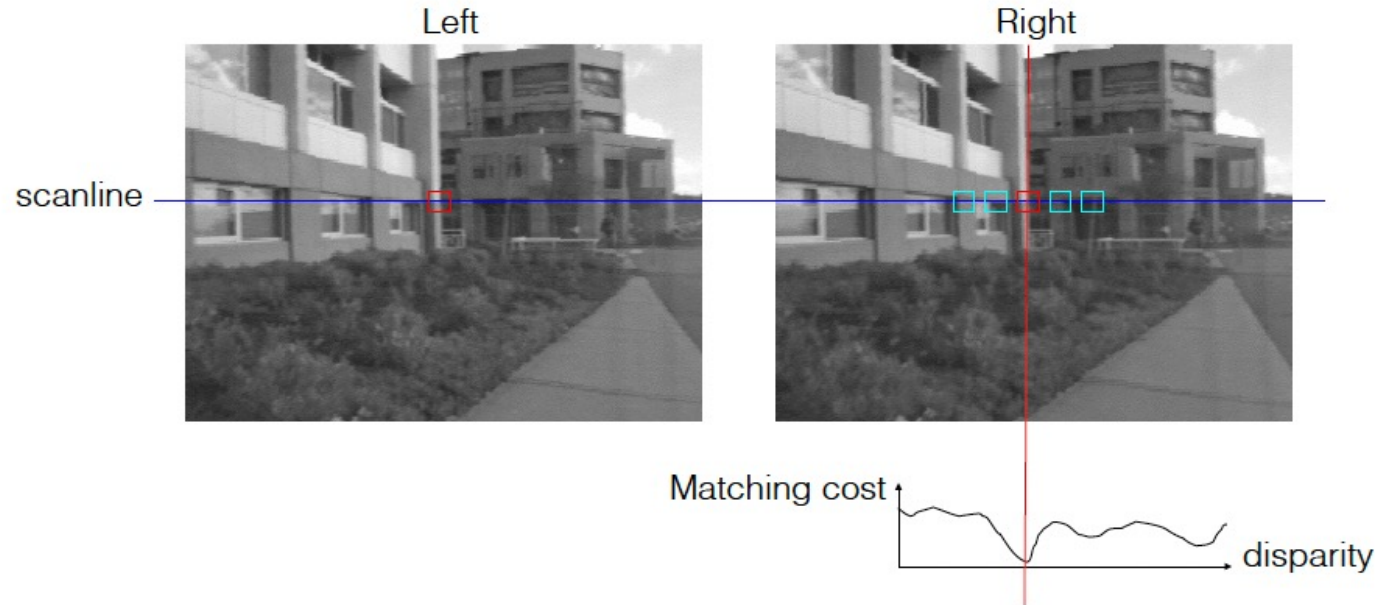
■  $Z = f \frac{T}{d}$



# Stereo Matching: Block Matching (BM)

166

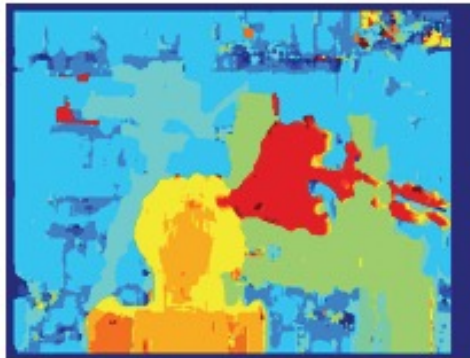
- ❑ Slide a window along the epipolar line
  - Compare contents of that window with the reference window in the left image
  - Find the best matched position
  - Matching cost: SSD or Normalized correlation (Like Template Matching)



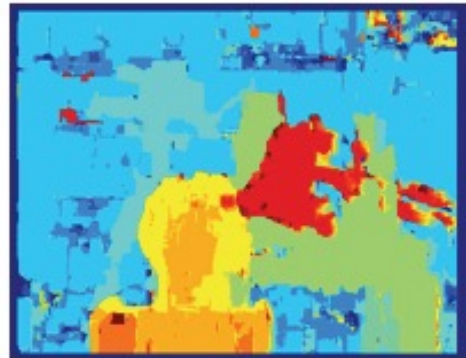
# Stereo Matching: Block Matching (BM)

167

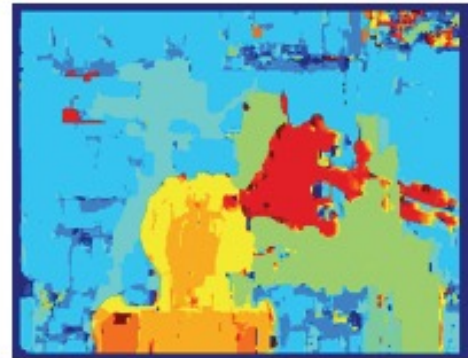
- ❑ Slide a window along the epipolar line
  - Compare contents of that window with the reference window in the left image
  - Find the best matched position
  - Matching cost: SSD or Normalized correlation (Like Template Matching)



SAD



SSD



NCC



Ground truth

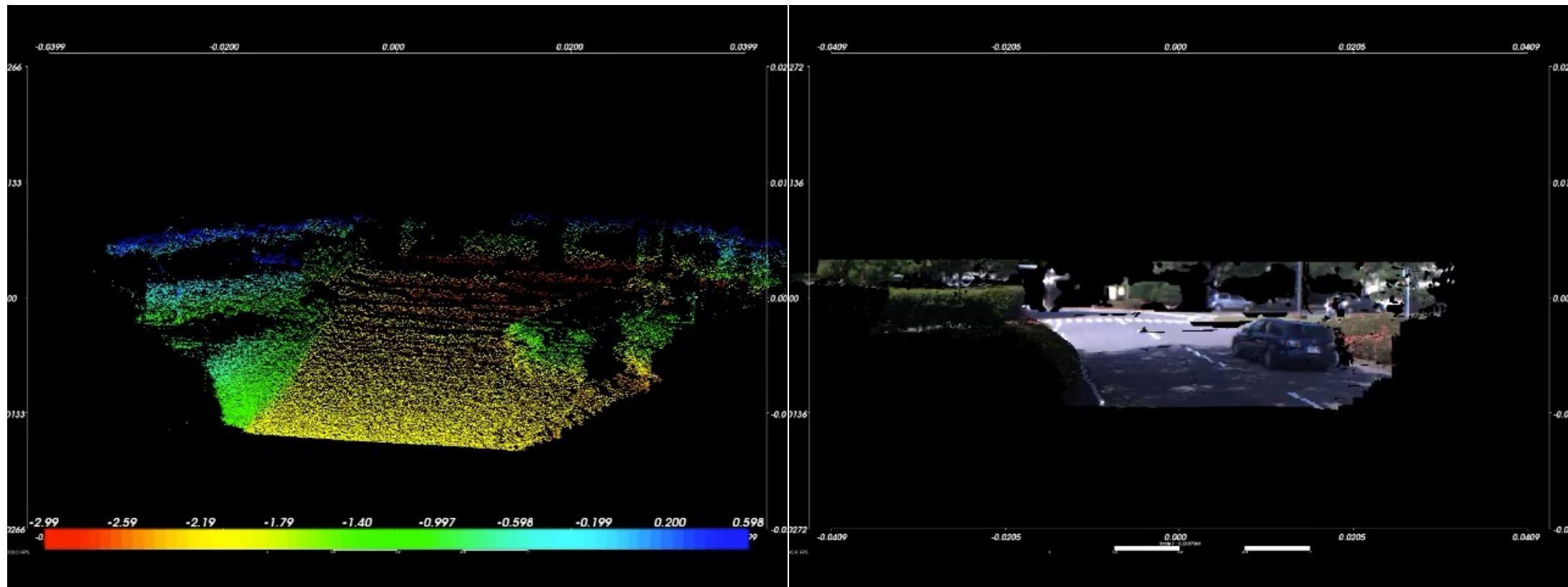
- ❑ Semi-Global Block Matching (SGBM)
  - Block Matching (Matching quality) + Smoothness cost



# Stereo Vision for Autonomous Driving

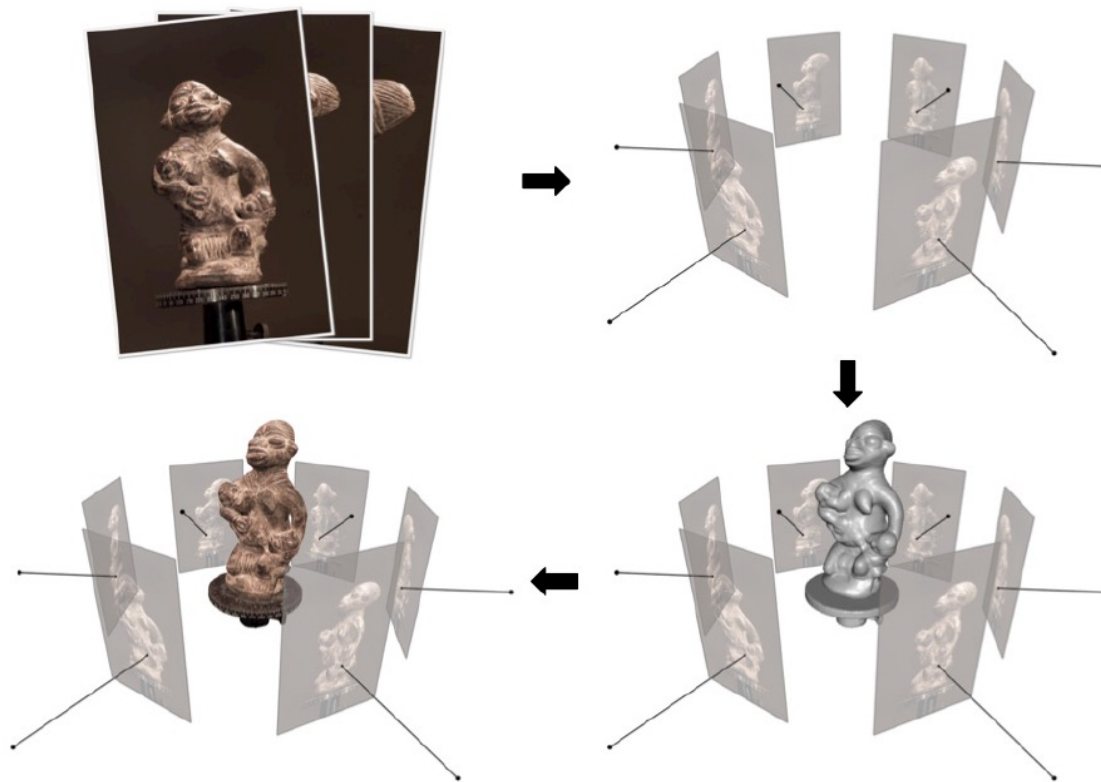
## 3D Reconstruction

168

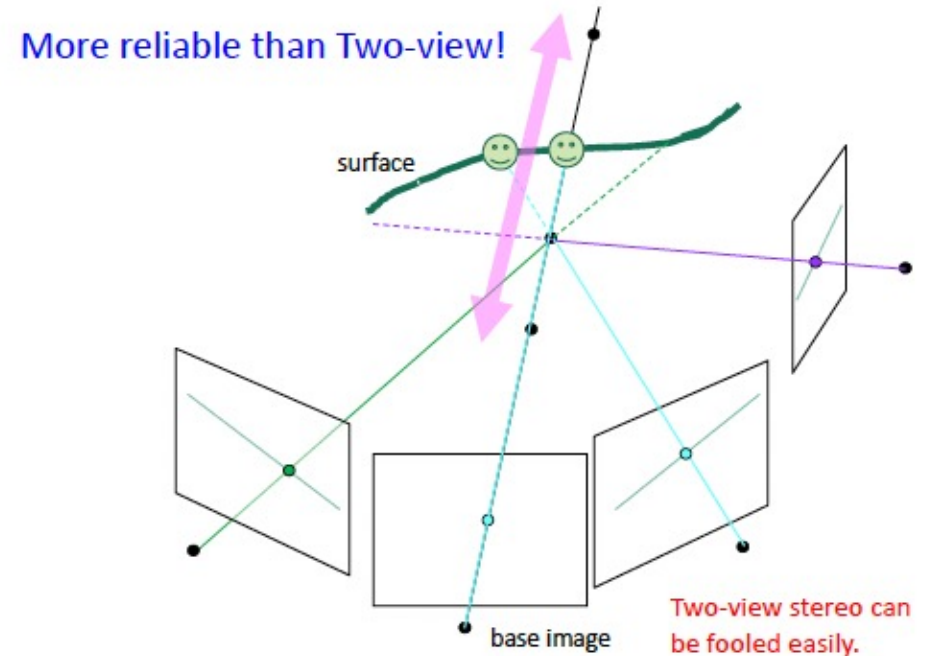


# Multi View Stereo

169



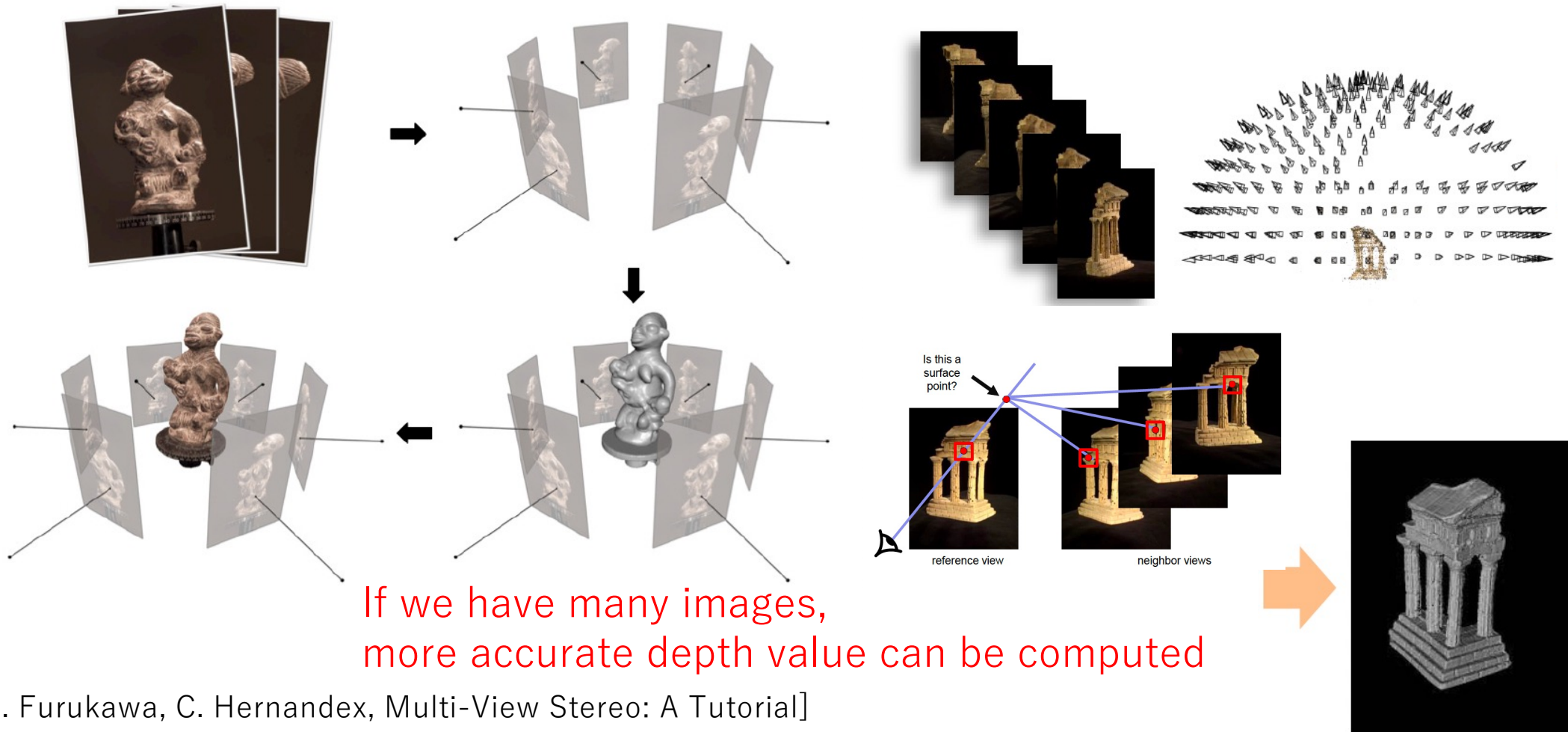
If we have many images,  
more accurate depth value can be computed



[Y. Furukawa, C. Hernandex, Multi-View Stereo: A Tutorial]

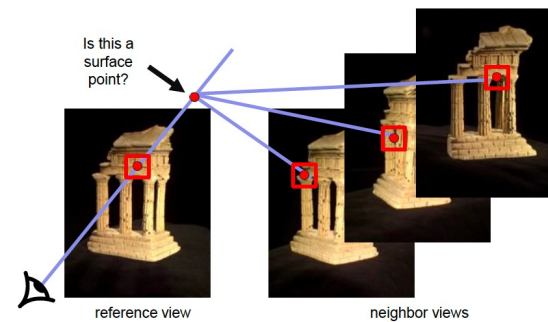
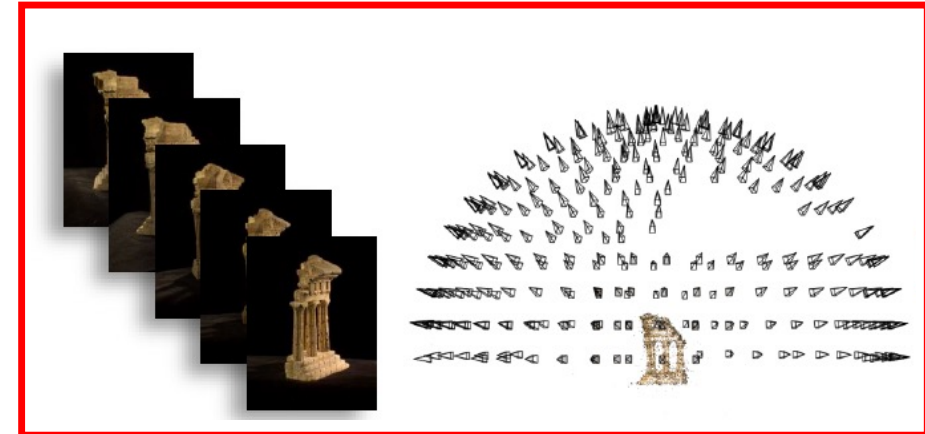
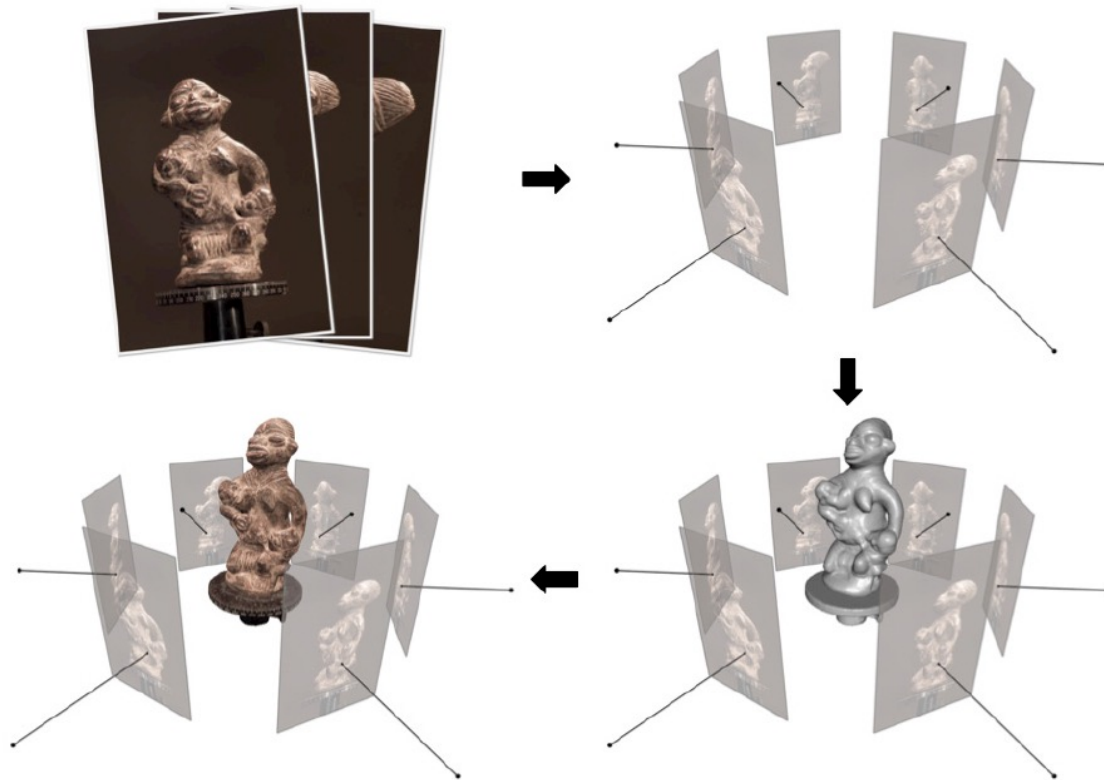
# Multi View Stereo

170



# Multi View Stereo

171



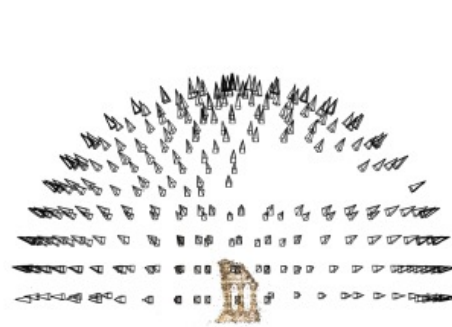
How to get camera position for each images?  
→ Structure From Motion

[Y. Furukawa, C. Hernandex, Multi-View Stereo: A Tutorial]

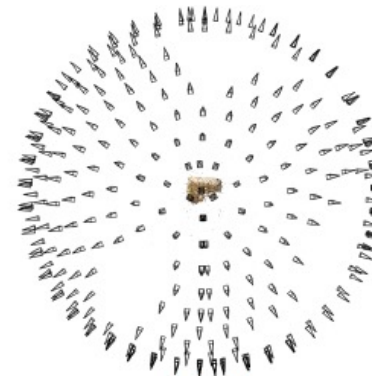


# Structure From Motion

172

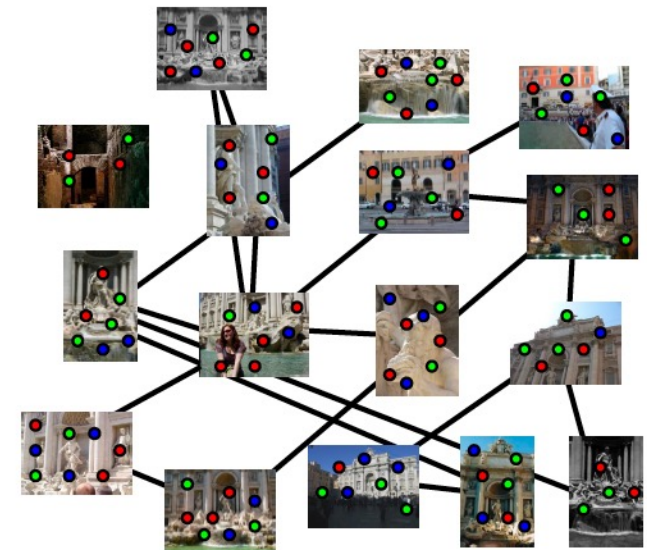


Reconstruction (side)

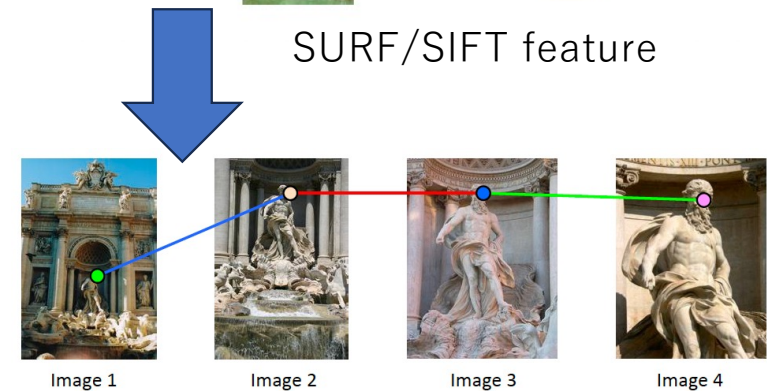


(top)

- Input: images with points in correspondence  
 $p_{i,j} = (u_{i,j}, v_{i,j})$
- Output
  - structure: 3D location  $\mathbf{x}_i$  for each point  $p_i$
  - motion: camera parameters  $\mathbf{R}_j, \mathbf{t}_j$  possibly  $\mathbf{K}_j$
- Objective function: minimize *reprojection error*

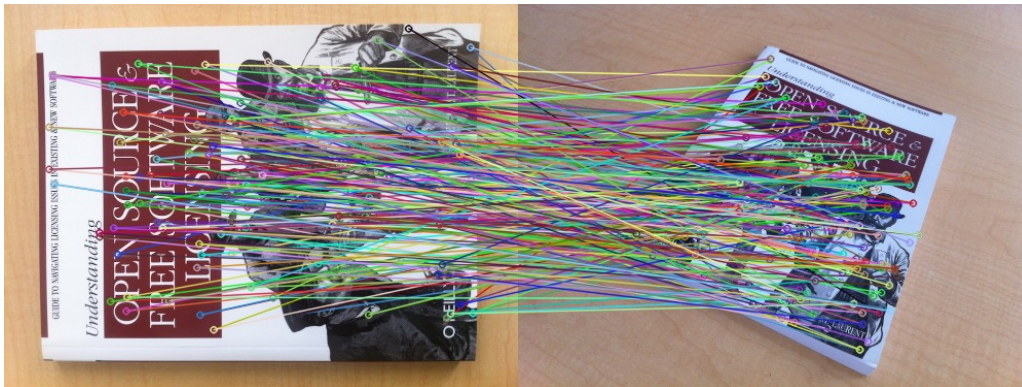


SURF/SIFT feature



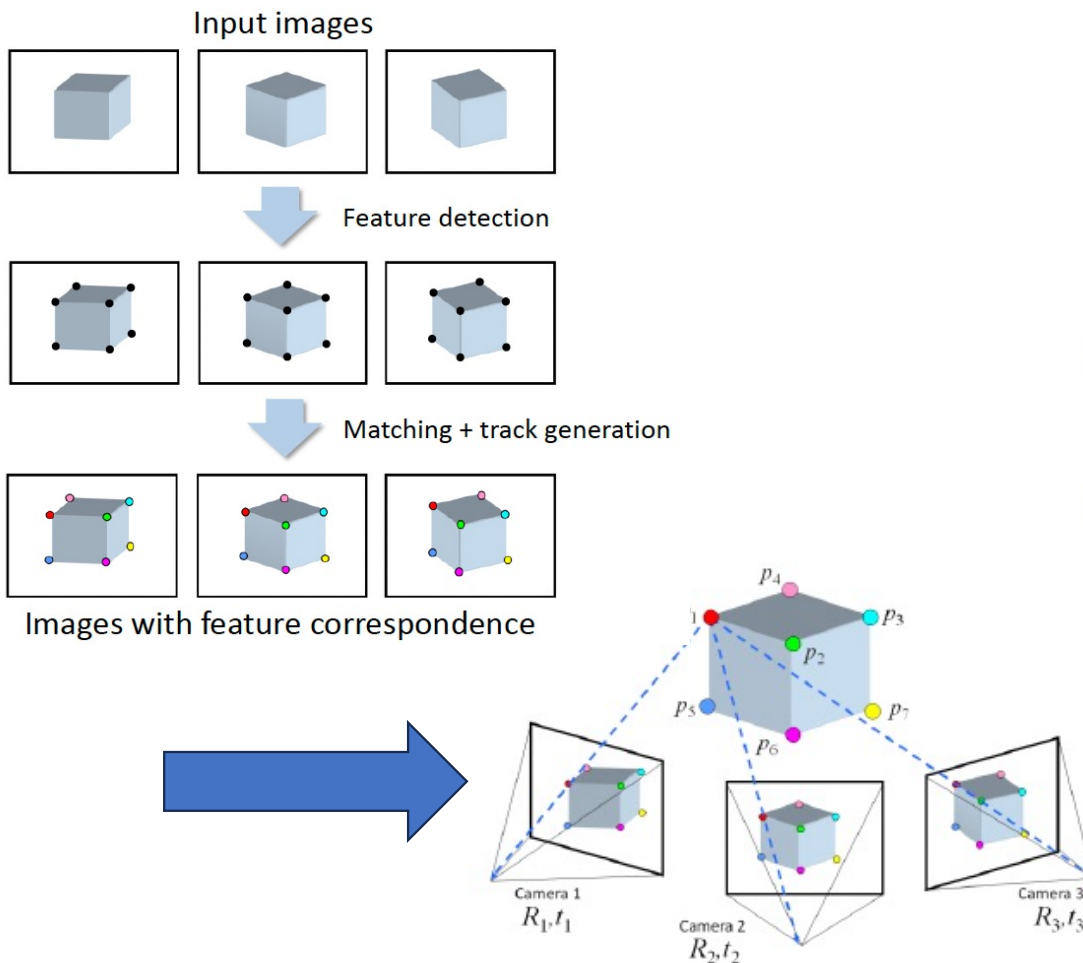
# Local Feature Matching

173

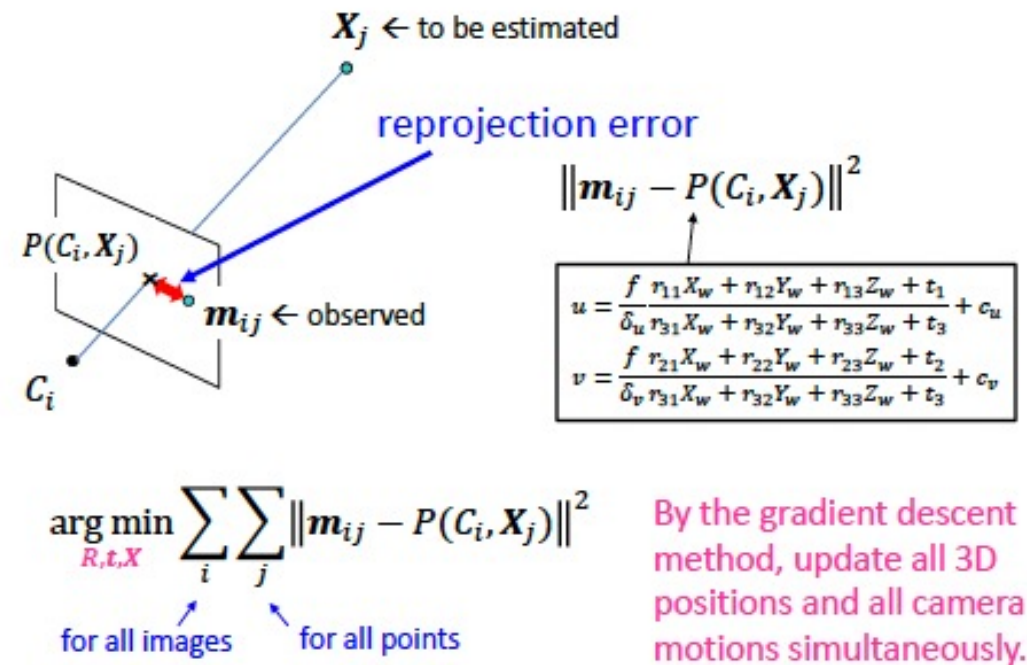


# Structure From Motion

174

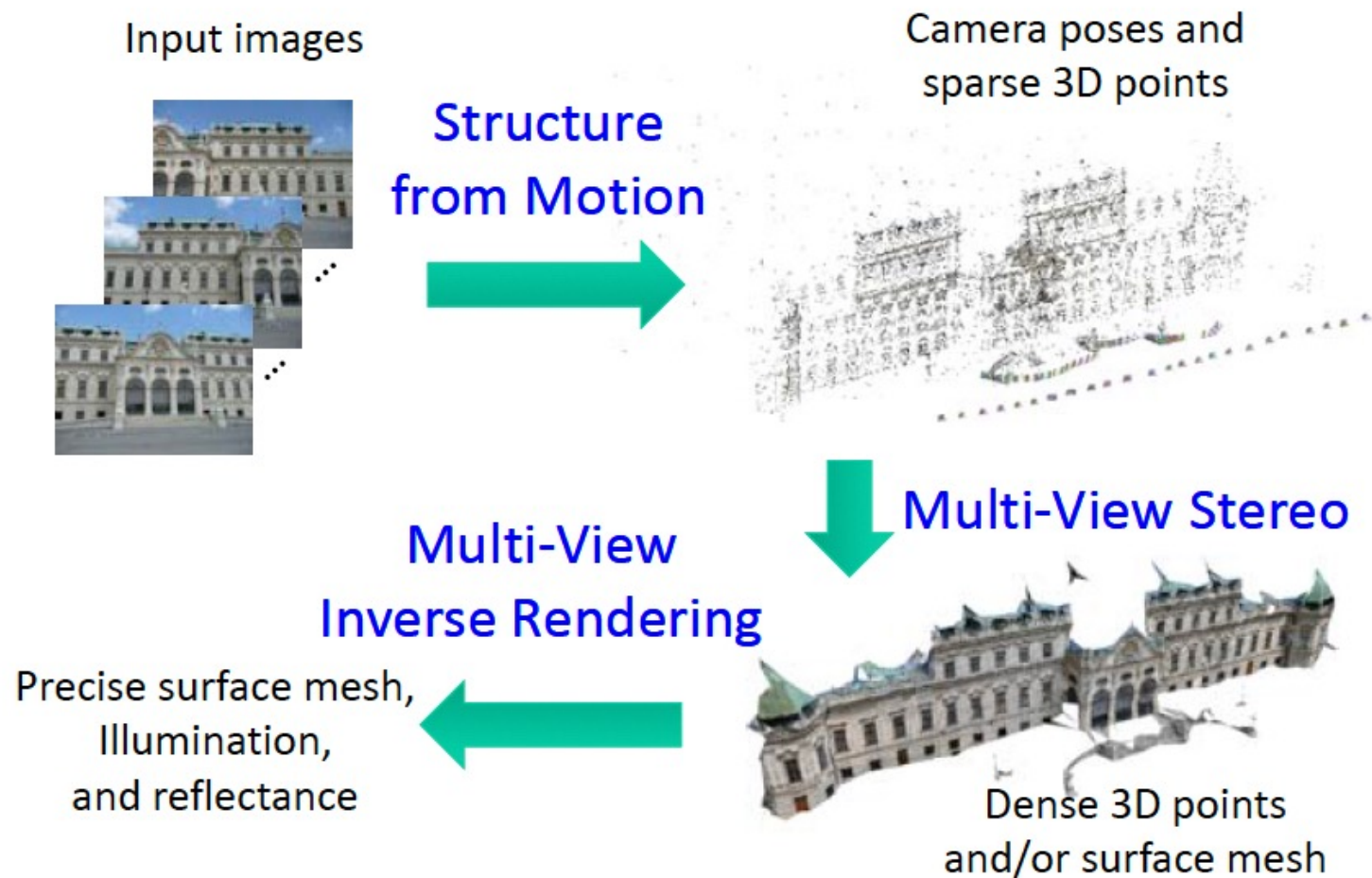


## Bundle Adjustment



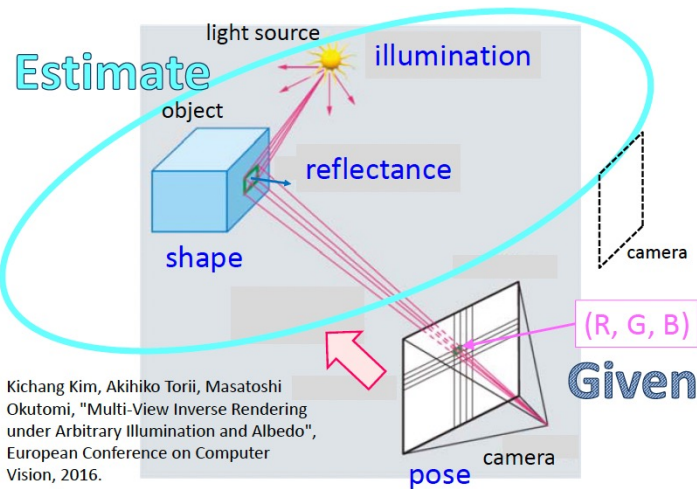
# SfM + MVS

175



# SfM + MVS + Inverse Rendering

176



Input images



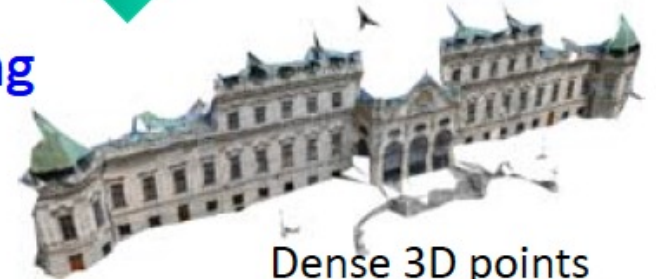
Structure  
from Motion



Camera poses and  
sparse 3D points



Multi-View  
Stereo

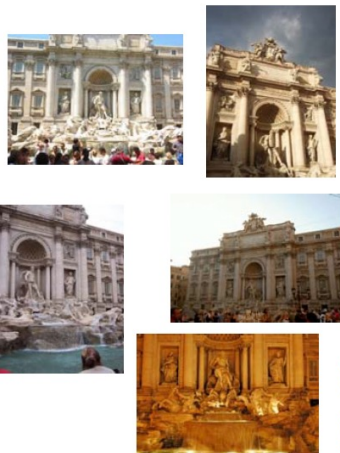


Dense 3D points  
and/or surface mesh

Multi-View  
Inverse Rendering



Precise surface mesh,  
illumination,  
and reflectance



Camera Poses  
Illumination  
Object Shape  
Reflectance

Kichang Kim, Akihiko Torii, Masatoshi Okutomi, "Multi-View Inverse Rendering under Arbitrary Illumination and Albedo", European Conference on Computer Vision, 2016.

# ○ NeRF (Neural Radiance Field) (1/3)

---

177

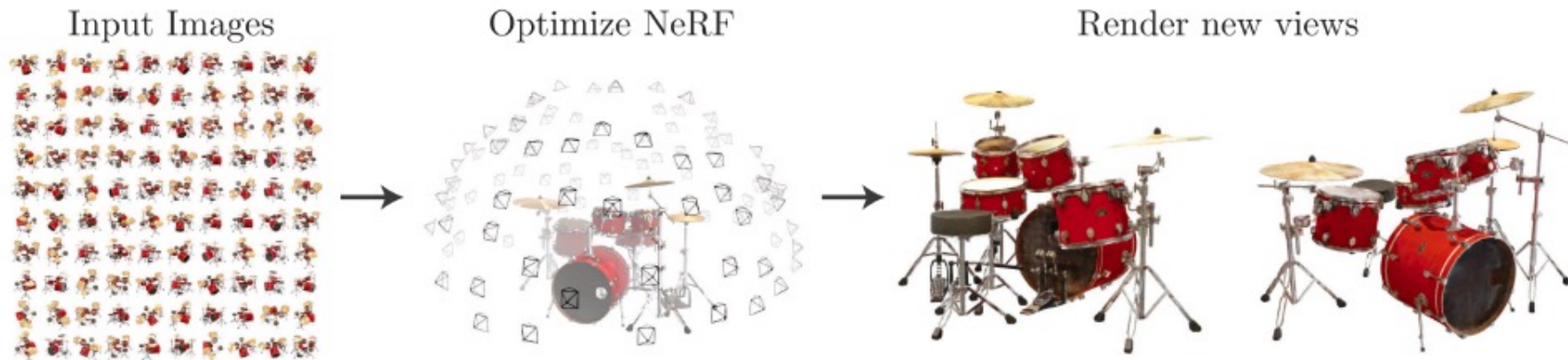


---

B. Midenhall et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis", arxiv preprint, 2020.

# ○ NeRF (Neural Radiance Field) (2/3)

178

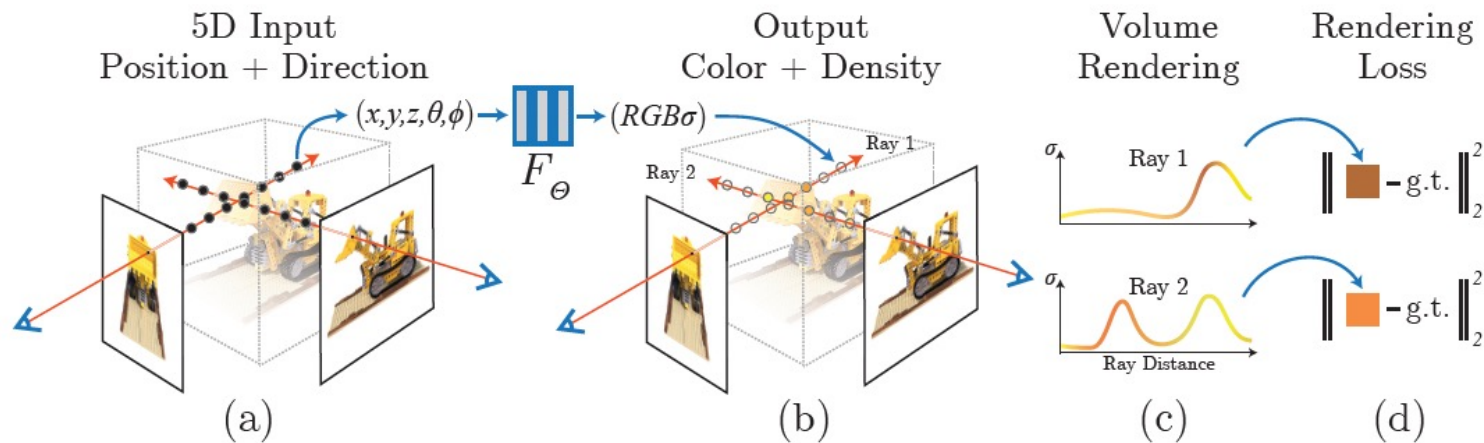


## □ Training image view

- Images with camera positions
- Camera positions are computed using Structure-from-Motion (SfM)

# NeRF (Neural Radiance Field) (3/3)

179



## □ Training image view

- Images with camera positions
- Camera positions are computed using Structure-from-Motion (SfM)

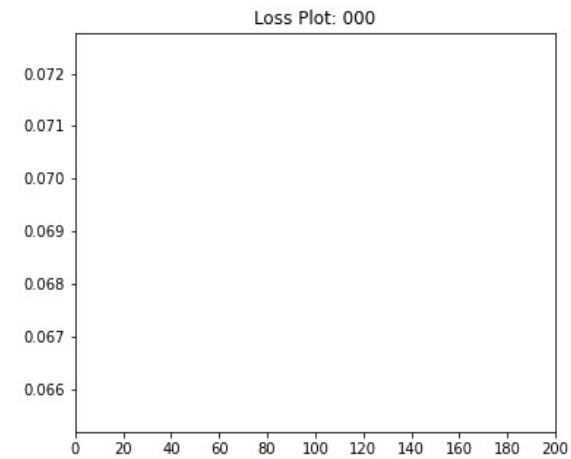
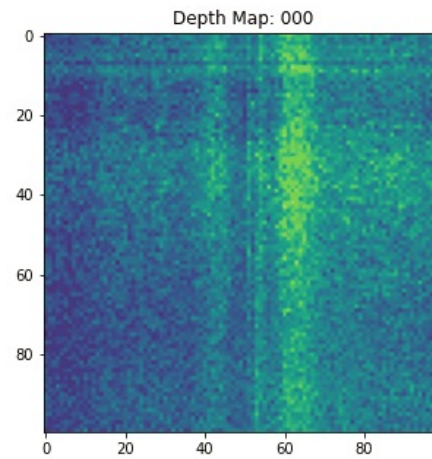
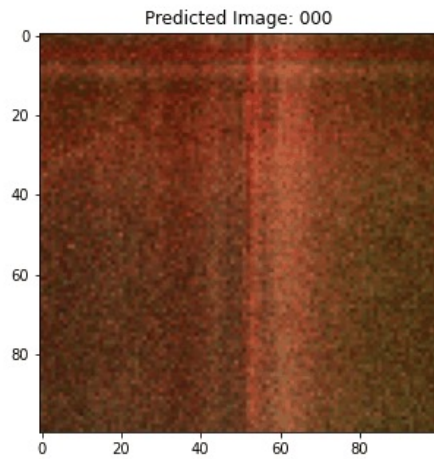
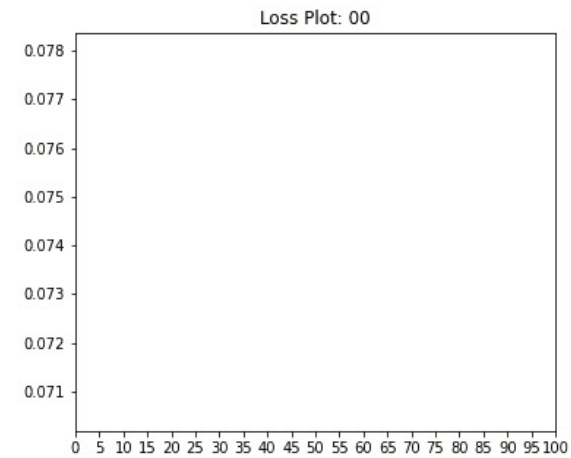
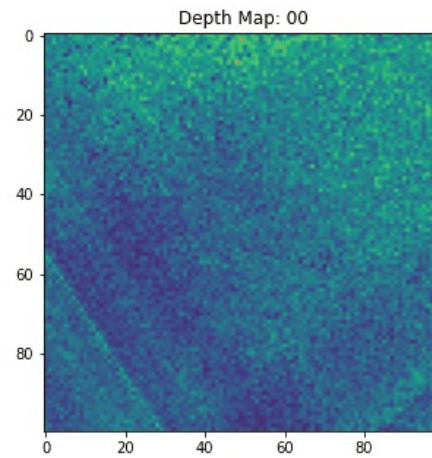
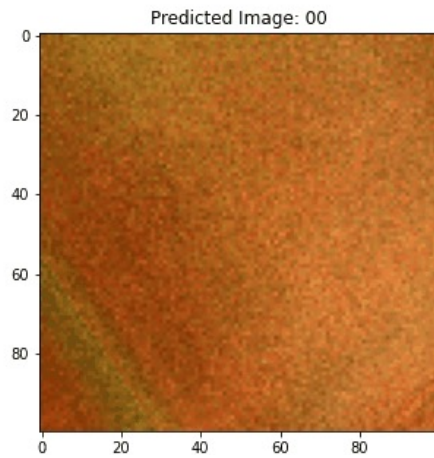
## □ Inference

- Input: 3d position and direction
- Output: RGB color and density

B. Mildenhall et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis", arxiv preprint, 2020.

# Training NeRF Model

180



<https://keras.io/examples/vision/nerf/>

# Latest Works for NeRF: Low Computation, High quality

181



## PlenVDB: Memory Efficient VDB-Based Radiance Fields for Fast Training and Rendering

Han Yan<sup>1</sup>, Celong Liu<sup>2</sup>, Chao Ma<sup>1†</sup>, Xing Mei<sup>2†</sup>

<sup>1</sup>MoE Key Lab of Artificial, AI Institute, Shanghai Jiao Tong University <sup>2</sup>ByteDance Inc.



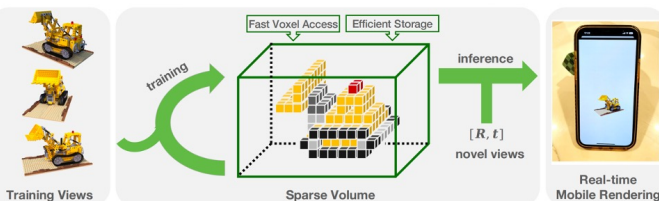
### Introduction

#### Challenges

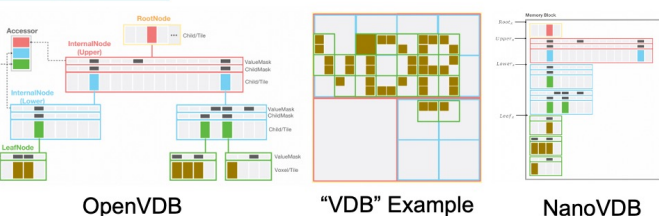
- NeRF is slow to train and render due to the heavy MLP.
- Find an **alternative data structure** is one direction.



#### Pipeline



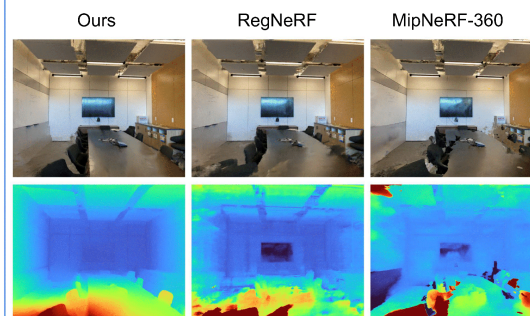
#### VDB



#### Contribution

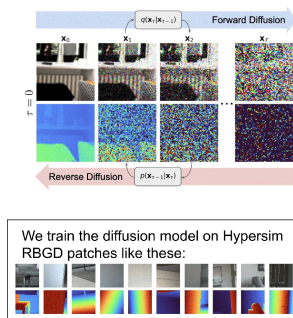
- We first use VDB as the sparse volume data structure for **NeRF acceleration**, and achieve **fast rendering** even on mobile devices.
- We propose a strategy that learns the VDB directly and achieves **fast training** and occupies **small storage**.

### Overview

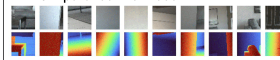


(Test views for scenes with 3 training views)

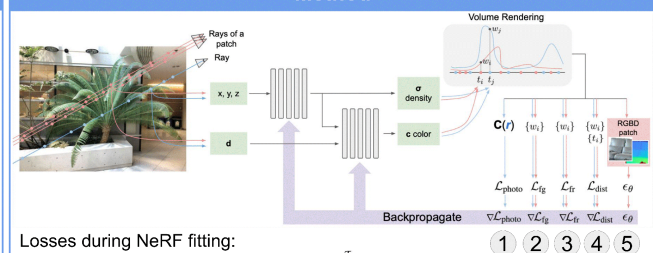
- NeRFs perform poorly with few training views
- We use a diffusion model trained on RGBD patches to regularize the NeRF during fitting
- We also use the distortion loss from Mip-NeRF 360 to encourage well-defined geometry
- Our method produces more plausible geometry than our baselines



We train the diffusion model on Hypersim RGBD patches like these:



### Method



Losses during NeRF fitting:

- Usual NeRF photometric loss:  $\mathcal{L}_{photo}(\sigma, c) = \sum_{i=1}^T ||I_i - C_i||_2$ .
- Encourages rays to be fully absorbed:  $\mathcal{L}_{lg} = \left(1 - \sum_{i=1}^N w_i\right)^2$
- Penalises density that is only visible from one camera frustum:  $\mathcal{L}_{fr} = \sum_i w_i 1(n_i \leq 1)$
- Mip-NeRF 360 distortion loss; encourages well-defined geometry  $\mathcal{L}_{dist} = \frac{1}{D(r)} \left( \sum_{i,j} w_i w_j \left| \frac{t_i + t_{j+1}}{2} - \frac{t_j + t_{i+1}}{2} \right| + \frac{1}{3} \sum_{i=1}^N w_i^2 (t_{i+1} - t_i) \right)$
- Encourages RGBD patches to have high likelihood according to the diffusion model:  $\epsilon_{\theta}(\mathbf{x}_T, \tau) \propto -\nabla_{\mathbf{x}} \log p(\mathbf{x})$

| Method                                                                                                                                                                                                       | LLFF   |        |        | DTU    |        |        | Chamfer-L1 ↓ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------|--------|--------|--------|--------|--------------|
|                                                                                                                                                                                                              | 3-view | 5-view | 9-view | 3-view | 5-view | 9-view |              |
| $\nabla \mathcal{L} = \nabla \mathcal{L}_{photo}$                                                                                                                                                            | 0.210  | 0.128  | 0.090  | 0.203  | 0.142  | 0.119  | 2.87         |
| $\nabla \mathcal{L} = \nabla \mathcal{L}_{photo} + \lambda_{lg} \nabla \mathcal{L}_{lg}$                                                                                                                     | 0.210  | 0.128  | 0.090  | 0.195  | 0.126  | 0.092  | 1.71         |
| $\nabla \mathcal{L} = \nabla \mathcal{L}_{photo} + \lambda_{lg} \nabla \mathcal{L}_{lg} + \lambda_{fr} \nabla \mathcal{L}_{fr}$                                                                              | 0.135  | 0.089  | 0.072  | 0.215  | 0.128  | 0.093  | 1.71         |
| $\nabla \mathcal{L} = \nabla \mathcal{L}_{photo} + \lambda_{lg} \nabla \mathcal{L}_{lg} + \lambda_{fr} \nabla \mathcal{L}_{fr} - \lambda_{DDM} \epsilon_{\theta}$                                            | 0.145  | 0.085  | 0.066  | 0.190  | 0.097  | 0.072  | 1.67         |
| $\nabla \mathcal{L} = \nabla \mathcal{L}_{photo} + \lambda_{lg} \nabla \mathcal{L}_{lg} + \lambda_{fr} \nabla \mathcal{L}_{fr} + \lambda_{dist} \nabla \mathcal{L}_{dist}$                                   | 0.118  | 0.071  | 0.060  | 0.185  | 0.092  | 0.056  | 1.36         |
| $\nabla \mathcal{L} = \nabla \mathcal{L}_{photo} + \lambda_{lg} \nabla \mathcal{L}_{lg} + \lambda_{fr} \nabla \mathcal{L}_{fr} + \lambda_{dist} \nabla \mathcal{L}_{dist} - \lambda_{DDM} \epsilon_{\theta}$ | 0.127  | 0.075  | 0.064  | 0.135  | 0.052  | 0.033  | 1.21         |
| DDM regularizer using 24x24 patches                                                                                                                                                                          | 0.126  | 0.074  | 0.061  | 0.195  | 0.068  | 0.043  | 1.22         |
| 24x24 patch DDM & NeRF fitted with $4 \times \lambda_{DDM}$                                                                                                                                                  | 0.129  | 0.074  | 0.062  | 0.260  | 0.080  | 0.050  | 1.22         |
| Patches from input images are not given to DDM                                                                                                                                                               | 0.139  | 0.078  | 0.066  | 0.159  | 0.063  | 0.049  | 1.91         |
| DDM trained with 20% of Hypersim scenes                                                                                                                                                                      | 0.132  | 0.078  | 0.066  | 0.163  | 0.057  | 0.035  | 1.65         |
| RGB-only DDM regularizer                                                                                                                                                                                     | 0.134  | 0.083  | 0.070  | 0.189  | 0.081  | 0.058  | 1.31         |
| $\tau = 0$ (no schedule) during NeRF fitting                                                                                                                                                                 | 0.137  | 0.081  | 0.067  | 0.152  | 0.055  | 0.042  | 1.31         |
| NeRF fitted with $4 \times \lambda_{DDM}$                                                                                                                                                                    | 0.146  | 0.088  | 0.076  | 0.220  | 0.134  | 0.071  | 2.56         |



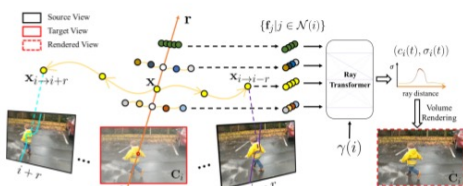
## DiffusioNeRF: Regularizing Neural Radiance Fields with Denoising Diffusion Models

Jamie Wynn and Daniyar Turmukhambetov



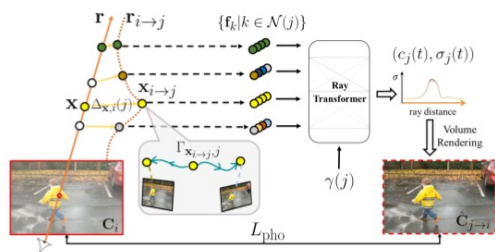
# Latest Works for NeRF: NeRF for Dynamic Scenes

182



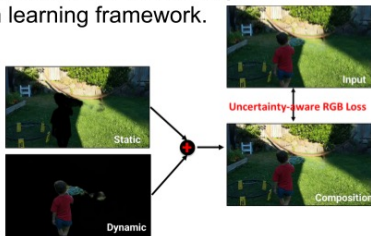
## Cross-time rendering for temporal consistency

- We render each frame at time  $i$  using the scene model from a nearby time  $j$ .
- A ray  $\mathbf{r}$  from image at  $i$  is rendered using aggregated features from curved ray  $\gamma_{i \rightarrow j}$  through ray transformer, which produces per-sample color and density at time  $j$ .



## Combining static and dynamic models

- We decompose scenes into static and dynamic components through a IBR-based motion segmentation module within a Bayesian learning framework.



Google Research



## DynIBaR: Neural Dynamic Image-Based Rendering

Award Candidate

Zhengqi Li<sup>1</sup>, Qianqian Wang<sup>1,2</sup>, Forrester Cole<sup>1</sup>, Richard Tucker<sup>1</sup>, Noah Snavely<sup>1</sup>  
<sup>1</sup>Google Research, <sup>2</sup>Cornell Tech



<https://dynibar.github.io/>